

Dictionaries

1. What are dictionaries and dictionary documentation

Dictionary documentation

Dictionaries are one of Python's *Built-in Types*

You find the documentation here among other built-in types (e.g. Numeric types, Iterator types, Set types, Text sequence types):

It is called a *Mapping Type* (since it maps from one value to another). [https:](https://python.readthedocs.io/en/latest/library/index.html)

[//python.readthedocs.io/en/latest/library/index.html](https://python.readthedocs.io/en/latest/library/index.html)

Then choose: 4.10. Mapping Types - dict

To reach: <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

- [Introduction](#)
 - [Notes on availability](#)
- [Built-in Functions](#)
- [Built-in Constants](#)
 - [Constants added by the site module](#)
- [Built-in Types](#)
 - [Truth Value Testing](#)
 - [Boolean Operations — and, or, not](#)
 - [Comparisons](#)
 - [Numeric Types — int, float, complex](#)
 - [Boolean Type - bool](#)
 - [Iterator Types](#)
 - [Sequence Types — list, tuple, range](#)
 - [Text and Binary Sequence Type Methods Summary](#)
 - [Text Sequence Type — str](#)
 - [Binary Sequence Types — bytes, bytearray, memoryview](#)
 - [Set Types — set, frozenset](#)
 - [Mapping Types — dict](#)
 - [Context Manager Types](#)
 - [Type Annotation Types — Generic Alias, Union](#)

2. When to use dictionaries

When to use dictionaries

The dictionary is a very useful data structure. You can use it for anything where you need to do a mapping between a *key* and an *value*.

The classic example is a word dictionary, where the key is a word you need to look up and the value is the translation of the word.

```
1 swedish_animal_words = {  
2     "hund" : "dog",  
3     "katt" : "cat",  
4     "svart" : "black",  
5     "brun" : "brown"}
```

When to use dictionaries

But an equally frequent usage is that you have a set of properties, which you want to assign values. The property (e.g. "name") is then the *key*, and the value is the *value* (e.g. "A collection of sustainability terms").

An authentic example of term list meta-data-file exported to the central terminology database Eurotermbank:

```
metadata_sustainability_terms =
```

```
{"name": "A collection of sustainability terms",
```

```
"description": "Hållbarhetstermgruppen: Termlista för hållbar utveckling | 2021",
```

```
"domainid": "52",
```

```
"attributionText": "Developed by the Institute for Language and Folklore (Institutet för språk och folkminnen, registration number: 22/0621)",
```

```
"cpEmail": "terminologi@isof.se",
```

```
"cpOrganization": "Hållbarhetstermgruppen",
```

```
"iprOrganization": "the Institute for Language and Folklore (Institutet för språk och folkminnen)",
```

```
"licence": "CC-BY-4.0",
```

```
"originalName": "Hållbarhetstermgruppen: Termlista för hållbar utveckling | 2021",
```

```
"originalNameLang": "sv",
```

```
"sourceURL": "https://rikstermbanken.isof.se" }
```

3. Dictionary syntax

Dictionary syntax

You create the dictionary with curly brackets, and separate the key and value with :

As for lists and tuples, each entry is separated by a comma. ✓

```
1 swedish_animal_words = {  
2     "hund" : "dog",  
3     "katt" : "cat",  
4     "svart" : "black",  
5     "brun" : "brown"}
```

Dictionary syntax

When looking up a key in the dictionary, you use square brackets, as follows:

```
1  swedish_animal_words = {  
2      "hund" : "dog",  
3      "katt" : "cat",  
4      "svart" : "black",  
5      "brun" : "brown"}  
6  
7  print (swedish_animal_words["hund"])  
8  print (swedish_animal_words["svart"])
```

dog

black

Dictionary syntax

You can also **add new key-value-pairs** to the dictionary, using square brackets:

```
1  swedish_animal_words = {  
2      "hund" : "dog",  
3      "katt" : "cat",  
4      "svart" : "black",  
5      "brun" : "brown"}  
6  
7  swedish_animal_words["kanin"] = "rabbit"  
8  print(swedish_animal_words)
```

```
{'hund': 'dog', 'katt': 'cat', 'svart': 'black',  
'brun': 'brown', 'kanin': 'rabbit'}
```

Dictionary syntax

You can also start with an empty dictionary, and then add keys and values one by one:

```
1 city_population_dictionary = {}  
2 city_population_dictionary["Stockholm"] = 978770  
3 city_population_dictionary["Madrid"] = 3305408  
4 city_population_dictionary["Berlin"] = 3677472  
5 print(city_population_dictionary)
```

'Stockholm': 978770, 'Madrid': 3305408, 'Berlin': 3677472

Dictionary syntax

Compare syntax for empty dictionary and empty set (This is a common mistake to confuse these)

```
1 city_population_dictionary = {}
2 city_population_dictionary["Stockholm"] = 978770
3 city_population_dictionary["Madrid"] = 3305408
4 city_population_dictionary["Berlin"] = 3677472
5 print(city_population_dictionary)
6
7 cities = set()
8 cities.add("Stockholm")
9 cities.add("Madrid")
10 cities.add("Berlin")
11 print(cities)
12
13 more_cities = {"Oslo", "Bergen"}
14 print(more_cities)
```

```
{'Stockholm': 978770, 'Madrid': 3305408, 'Berlin': 3677472}
{'Berlin', 'Madrid', 'Stockholm'}
{'Oslo', 'Bergen'}
```

4. Python built-in functions applied to dictionaries

Python built-in functions applied to dictionaries

len()

len() returns the number of key-value pairs in the dictionary.

```
1 city_population_dictionary = {}  
2 city_population_dictionary["Stockholm"] = 978770  
3 city_population_dictionary["Madrid"] = 3305408  
4 city_population_dictionary["Berlin"] = 3677472  
5 print(len(city_population_dictionary))
```

3

Python built-in functions applied to dictionaries

type()

`type()` returns the type of the variable (which in this case will be a dictionary)

```
1 city_population_dictionary = {}
2 city_population_dictionary["Stockholm"] = 978770
3 city_population_dictionary["Madrid"] = 3305408
4 city_population_dictionary["Berlin"] = 3677472
5 print(type(city_population_dictionary))
```

<class 'dict'>

5. Dictionary methods

Dictionary methods

keys() and values()

(Remember: *methods* are applied with the dot-notation.)

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5  
6 print(city_population_dictionary.keys())
```

dict_keys(['Stockholm', 'Madrid', 'Berlin'])

```
1  
2 print(city_population_dictionary.values())
```

dict_values([978770, 3305408, 3677472])

Dictionary methods

items()

With `items()`, you get tuples containing (key, value)

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5  
6 print(city_population_dictionary.items())
```

```
dict_items([('Stockholm', 978770), ('Madrid', 3305408), ('Berlin',  
3677472)])
```

Dictionary methods

It is often useful to loop over the items()

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5  
6 for el in city_population_dictionary.items():  
7     print(el)
```

('Stockholm', 978770)

('Madrid', 3305408)

('Berlin', 3677472)

Dictionary methods

You can also pick out the key and value directly when you loop over a dictionary using `items()`. `items()` will make it possible to use

two loop variables.

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5  
6 for city, population in city_population_dictionary.items():  
7     print(city + " has a population of: " + str(population))
```

Stockholm has a population of: 978770

Madrid has a population of: 3305408

Berlin has a population of: 3677472

6. Checking if something is in a dictionary

Checking if something is in a dictionary

With the keyword `in`,
you can check if something is in a dictionary.

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5 print('Stockholm' in city_population_dictionary)
```

True

```
1 print('Oslo' in city_population_dictionary)
```

False

```
1 print(city_population_dictionary["Oslo"])
```

Will result in an exception being raised, more specifically:
`KeyError: 'Oslo'`

Something is not in a dictionary

default value

You can use `get` if you want a default value when the key is not in the dictionary

```
1 city_population_dictionary = {  
2     'Stockholm': 978770,  
3     'Madrid': 3305408,  
4     'Berlin': 3677472}  
5 print(city_population_dictionary.get("Oslo", "Unknown"))
```

Unknown

```
1 print(city_population_dictionary.get("Stockholm", "Unknown"))
```

978770

7. What can be used as dictionary keys?

What can be used as dictionary keys?

Immutable types can be used as keys, e.g. strings, tuples numbers

```
1 travel_times [("Stockholm", "Warsaw")] = 23
2 city_population_dictionary["Stockholm"] = 978770
3 numbers[100] = "hundred"
```

But mutable types *cannot* be used.

```
1 travel_times[["Stockholm", "Warsaw"]] = 23
```

Will result in:

`TypeError: unhashable type: 'list'`

8. Associate several values
with a dictionary key

Associate several values with a dictionary key

Often, you need to associate several values to the same key. For instance, you might want to add several English translations to each Swedish word. Then you can use a list as a data structure to store the values, as follows:

```
1 swedish_animal_words = {  
2     "brun" : ["brown"],  
3     "kanin": ["rabbit"]}  
4  
5 print (swedish_animal_words)
```

'brun': ['brown'], 'kanin': ['rabbit']

You can then append more elements to the value in the dictionary. Since the value is a list:

```
1 swedish_animal_words["kanin"].append("bunny")  
2 print (swedish_animal_words)
```

'brun': ['brown'], 'kanin': ['rabbit', 'bunny']

Associate several values with a dictionary key

You can add several values to the list when creating the dictionary.

```
1  swedish_animal_words = {  
2      "brun" : ["brown"],  
3      "kanin": ["rabbit", "bunny"]}  
4  print(swedish_animal_words)
```

'brun': ['brown'], 'kanin': ['rabbit', 'bunny']

```
1 text = ('One morning, when Gregor Samsa woke from troubled ',
2         'dreams, he found himself transformed in his bed ',
3         'into a horrible vermin He lay on his armour-like ',
4         'back, and if he lifted his head a little he could ',
5         'see his brown belly, slightly domed and divided by ',
6         'arches into stiff sections')
```

```
7
8 def count_words(text):
9
10     words = text.replace(' ', '').lower().split()
11     counter = {}
12
13     for w in words:
14         if w not in counter:
15             counter[w] = 1
16         else:
17             counter[w] += 1
18             # This means: counter[w] = counter[w] + 1
19     return counter
20
21 word_freqs = count_words(text)
22 print(word_freqs)
23 print(word_freqs["and"])
```

{ 'one': 1, 'morning': 1, 'when': 1, 'gregor': 1,
'samsa': 1, 'woke': 1, 'from': 1, 'troubled': 1,
'dreams': 1, 'he': 4, 'found': 1, 'himself': 1,
'transformed': 1, 'in': 1, 'his': 4, 'bed': 1,
'into': 2, 'a': 2, 'horrible': 1, 'vermin': 1,
'lay': 1, 'on': 1, 'armour-like': 1, 'back': 1,
'and': 2, 'if': 1, 'lifted': 1, 'head': 1,
'little': 1, 'could': 1, 'see': 1, 'brown': 1,
'belly': 1, 'slightly': 1, 'domed': 1,
'divided': 1, 'by': 1, 'arches': 1,
'stiff': 1, 'sections': 1 }